

TEXTBOOK EDITION • BACKEND ENGINEERING & RUNTIME

JavaScript & Node.js Runtime Internals

V8 Engine, Execution Context, libuv 6-Phase Event Loop & Scalable Backend

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	14 Hours
DIFFICULTY LEVEL	Candidate Superpower
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — JavaScript Event Loop, V8 Call Stack & Microtask Scheduling

Study Time: 35m

Chapter 02 — Most Asked & Tricky JavaScript Interview Questions

Study Time: 35m

Chapter 03 — Node.js Event Loop, libuv & Concurrency Internals

Study Time: 35m

Chapter 04 — Node.js & Backend Architecture Last-Minute Cheatsheet

Study Time: 35m

CHAPTER 01

JavaScript Event Loop, V8 Call Stack & Microtask Scheduling

PRIORITY

● High Priority

DIFFICULTY

Candidate Superpower

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

JavaScript Event Loop, V8 Call Stack & Microtask Scheduling

1. Why This Matters

While Node.js integrates libuv for operating system interactions, JavaScript itself relies on the **V8 Engine** for memory management, the call stack, and microtask scheduling. In IDFC FIRST Bank technical interviews, senior engineers test whether you can predict the exact asynchronous execution sequence of complex nested Promises, async/await blocks, and microtasks, and understand how the browser and Node runtime environments manage non-blocking concurrency.

2. Prerequisites

- JavaScript execution contexts, call stack, and hoisting.
- Promises, `then()` chaining, and `async/await` syntax.

3. Concept

The JavaScript Concurrency Model

JavaScript is a **single-threaded non-blocking** language with a run-to-completion model. A single Call Stack can only execute one statement at any instant. Asynchronous operations do not execute in parallel on the call stack; their callbacks are queued and deferred.

flowchart TD

```
CS["Call Stack (V8 Engine)\nRuns current synchronous frame until completion"]
MT["Microtask Queue\n- process.nextTick (Node)\n- Promise.then / catch / finally\n- queueMicrotask\n(DRAINED FULLY AFTER EVERY SYNCHRONOUS TASK)"]
Mac["Macrotask Queue / Task Queue\n- setTimeout / setInterval\n- setImmediate (Node)\n- I/O events\n(Processed ONE by ONE between microtask drains)"]
```

```
CS → |Schedules| MT
CS → |Schedules| Mac
MT → |Yields back to| CS
Mac → |Yields back to| CS
```

4. Simple Example: The Microtask Priority Queue

```
console.log('A');

setTimeout(() => {
  console.log('B');
}, 0);

Promise.resolve()
  .then(() => {
    console.log('C');
    return Promise.resolve('D');
  })
  .then((val) => {
    console.log(val);
  });

console.log('E');
```

/
Output:
A
E
C
D
B
/

5. Real-World Banking Example: Atomic Transaction State Transition

In a banking frontend or Node API gateway handling fund transfers, updating UI state or dispatching background analytics must not interrupt transactional balance verification: - State validation and atomic balance checks run synchronously or via **microtasks** (`queueMicrotask` / `Promise`), ensuring they finish *before* the browser repaints the DOM or before the network macrotask queue yields to lower-priority I/O.

6. Code: Tricky Interview Async/Await Execution Order

```
async function async1() {
  console.log('async1 start');
  await async2();
  console.log('async1 end'); // Scheduled into Microtask Queue!
}

async function async2() {
  console.log('async2');
}

console.log('script start');

setTimeout(() => {
  console.log('setTimeout');
}, 0);

async1();

new Promise((resolve) => {
  console.log('promise constructor');
  resolve();
}).then(() => {
  console.log('promise then');
});

console.log('script end');

/*
Complete Output Sequence:
1. script start
2. async1 start
3. async2
4. promise constructor
5. script end
6. async1 end
7. promise then
8. setTimeout
*/
```

7. How It Works Internally: Async/Await Desugaring

Under the hood, `async/await` is syntactic sugar built on top of **Generators** and **Promises**. When V8 encounters:

```
async function foo() {
  console.log('Before');
  await bar();
  console.log('After');
}
```

V8 conceptually desugars this into:

```
function foo() {
  console.log('Before');
  return Promise.resolve(bar()).then(() => {
    console.log('After');
  });
}
```

- When `await bar()` is called, `bar()` executes synchronously until its first asynchronous suspension point.
- The remaining body of `foo()` is wrapped inside an implicit `.then()` handler and pushed into the **Promise Microtask Queue**.
- The execution context of `foo` is suspended, popping off the call stack and allowing the rest of the synchronous script to continue running.

8. Common Mistakes

1. **Believing Promise Executor Functions are Asynchronous:** `javascript new Promise((resolve) => { console.log('Sync!'); // This runs SYNCHRONOUSLY and IMMEDIATELY! resolve(); });` Only the callbacks attached via `.then()`, `.catch()`, or `.finally()` are scheduled asynchronously into the microtask queue.
2. **Causing Starvation with Infinite Microtasks:** `javascript function infiniteMicrotasks() { Promise.resolve().then(infiniteMicrotasks); } infiniteMicrotasks(); // The engine will NEVER execute setTimeout or render UI frames!` Because the microtask queue must be completely emptied before any macrotask (like `setTimeout` or UI rendering) is processed, an infinite microtask chain permanently locks the thread.

9. Performance / Complexity Matrix

Task Type	Queue	Processing Frequency	Starves Event Loop if Infinite?
<code>Promise.then()</code>	Microtask	Drained completely before next macrotask	YES
<code>queueMicrotask()</code>	Microtask	Drained completely before next macrotask	YES
<code>process.nextTick()</code>	Process Microtask	Highest priority; drains before Promises	YES
<code>setTimeout()</code>	Macrotask	One task per loop tick	No (allows microtasks in between)
<code>setImmediate()</code>	Macrotask	One task per loop tick (Check phase)	No

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is the difference between a microtask and a macrotask in JavaScript? Give two examples of each.

Medium

- **Q:** Why does `new Promise((resolve) => { console.log('1'); resolve(); })` log `'1'` before code that appears after the Promise declaration?

Hard

- **Q:** Explain what happens when an unhandled Promise rejection occurs in Node.js. How has the runtime behavior evolved from Node.js 14 to Node.js 16+, and what is the effect on process termination?

11. Follow-up Questions from Interviewer

- "In a browser, when does UI rendering (RequestAnimationFrame / DOM repaint) occur relative to microtasks and macrotasks?" (Answer: Rendering occurs after the microtask queue has completely drained, but before the next macrotask is dequeued).
- "What is `queueMicrotask()` and when should a developer use it instead of `Promise.resolve().then()`?"

12. Model Answer: Microtask Queue Starvation

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "If you have a recurring task that needs to run continuously, why should you schedule it with `setImmediate` or `setTimeout` rather than recursive `Promise.then`?"

Model Answer: "The JavaScript runtime enforces a strict rule: **the Microtask Queue must be 100% empty before the event loop can advance to any subsequent Macrotask or I/O phase.**

If a recurring task reschedules itself via `Promise.resolve().then()`, new microtasks are continuously appended to the microtask queue faster than or equal to the rate they are consumed.

Because the microtask queue never empties: 1. Timers (`setTimeout`) never fire. 2. Network I/O events cannot be processed. 3. In the browser, layout and painting steps are starved, freezing the user interface entirely.

By scheduling recursive operations using `setImmediate` (in Node) or `setTimeout` (in the browser), the callback is placed in the **Macrotask Queue**. The event loop executes *one* macrotask, allows microtasks, I/O polling, and UI rendering to breathe, and preserves runtime responsiveness."

13. Practical Exercise

Run this code in Node.js and predict whether `console.log('Timer fired')` will ever execute:

```
let counter = 0;

setTimeout(() => {
  console.log('Timer fired!');
}, 50);

function scheduleNext() {
  if (counter++ < 1000000) {
    queueMicrotask(scheduleNext);
  }
}

scheduleNext();
// Notice: The 50ms timer is delayed until all 1,000,000 microtasks finish!
```

14. Quick Revision

- Synchronous call stack code runs first to completion.
- Microtasks (`Promise`, `queueMicrotask`) execute immediately when the call stack clears.
- Macrotasks (`setTimeout`, `I/O`) execute one at a time between microtask queue flushes.
- Promise constructors run synchronously; only `.then()` handlers are deferred.
- Infinite microtask recursion starves timers, I/O, and rendering.

15. Interview Checklist

- ☐ Correctly traces nested `async/await` and Promise code output.
- ☐ Explains how `async/await` desugars into Promises and generator continuations.
- ☐ Understands the starvation risk of recursive microtasks.
- ☐ Explains when DOM rendering occurs relative to the task queues.



CHAPTER 01 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- JavaScript execution contexts, call stack, and hoisting.
- Promises, `.then()` chaining, and `async/await` syntax.
- State validation and atomic balance checks run synchronously or via microtasks (`'queueMicrotask' / 'Promise'`), ensuring



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

Most Asked & Tricky JavaScript Interview Questions

PRIORITY

● High Priority

DIFFICULTY

Candidate Superpower

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Most Asked & Tricky JavaScript Interview Questions

The Senior Engineer Playbook: V8 Internals, Event Loop, Closures, Coercion & FinTech Traps

 BANKING & FINTECH ARCHITECTURE SCENARIO**Target Role:** Senior / Core Developer (3+ Years Experience) | Strategic Projects, Bengaluru**Prerequisites:** Modern ES6+, JavaScript Runtime Architecture, Asynchronous Programming.**Key Focus:** Code output brain-teasers, edge cases, V8 deoptimization pitfalls, and real-world banking bugs. **Quick Navigation**

1. [The Microtask vs Macrotask Event Loop Execution Order](#)
2. [Scoping, Hoisting & The Temporal Dead Zone \(TDZ\)](#)
3. [The Classic `var` Inside `setTimeout` Loop Trap](#)
4. [this Keyword Binding Rules & Arrow Function Nuances](#)
5. [Type Coercion Brain-Teasers & The IEEE 754 Currency Trap](#)
6. [Prototypal Inheritance & `\_\_proto\_\_` vs `prototype`](#)
7. [Promise Concurrency: `all` vs `allSettled` vs `race` vs `any`](#)
8. [Object Mutation, Shallow vs Deep Copy & `structuredClone`](#)
9. [Writing Production Debounce & Throttle from Scratch](#)
10. [V8 Engine Optimization: Hidden Classes & Inline Caches](#)

1. The Microtask vs Macrotask Event Loop Execution Order

 Why Interviewers Ask This

To verify whether you truly understand asynchronous concurrency in JavaScript, or simply rely on "asynchronous means runs later". In high-throughput banking systems (e.g. processing payment webhooks), microtask starvation can completely freeze UI rendering or network I/O.

🧩 Tricky Code Puzzle

What is the exact console output order of the following snippet?

```
console.log('1: Script Start');

setTimeout(() => {
  console.log('2: setTimeout 0');
}, 0);

Promise.resolve()
  .then(() => {
    console.log('3: Promise 1');
  })
  .then(() => {
    console.log('4: Promise 2');
  });

queueMicrotask(() => {
  console.log('5: queueMicrotask');
});

process.nextTick?.(() => {
  console.log('6: process.nextTick');
});

console.log('7: Script End');
```

❌ Intuitive (Wrong) Answer

"1, 7, 2, 3, 4, 5, 6 — because `setTimeout` was scheduled first with 0 delay, followed by promises and `nextTick`."

✅ Actual Correct Answer & Execution Trace

```
1: Script Start
7: Script End
6: process.nextTick (Node.js environment)
3: Promise 1
5: queueMicrotask
4: Promise 2
2: setTimeout 0
```

🧠 Execution Breakdown & Hinglish Mental Model

🔑 KEY ENGINEERING INSIGHT

"JavaScript ka execution 3 layers me chalta hai: 1. **Call Stack (Synchronous Code)**: Jo code seedha execute hota hai wo stack me run hota hai (`1: Script Start` , fir `7: Script End`). 2. **Microtask Queue (High Priority VIP Lounge)**: Isme `process.nextTick` (highest priority in Node.js), `Promise.then/catch/finally` , aur `queueMicrotask()` aate hain. Call stack empty hote hi Event Loop pehle Microtask queue ko COMPLETELY drain karta hai. 3. **Macrotask / Task Queue (General Public)**: `setTimeout` , `setInterval` , `setImmediate` , I/O callbacks. Ye tabhi chalte hain jab saare microtasks khatam ho chuke hon!"

1. Synchronous statements execute: `1: Script Start` and `7: Script End` .

2. Stack becomes empty. Event Loop checks Microtask Queue.
3. In Node.js, `process.nextTick` queue runs first: `6: process.nextTick`.
4. Promise 1 callback runs: `3: Promise 1`. It schedules Promise 2 onto the microtask queue.
5. Next microtask already waiting is `5: queueMicrotask`.
6. Promise 2 callback runs: `4: Promise 2`.
7. Microtask queue is now completely exhausted.
8. Event loop proceeds to Macrotask phase (Timers): `2: setTimeout 0`.

FinTech Production Consequence

If a recursive Promise or unbounded microtask loop is introduced (e.g. an aggressive retry loop during payment processing), the Macrotask queue will **starve**. Incoming network I/O, heartbeat pings, and timer timeouts will NEVER execute, leading to connection dropouts and false-positive service failovers.

2. Scoping, Hoisting & The Temporal Dead Zone (TDZ)

Why Interviewers Ask This

Tests your knowledge of lexical environments, Variable Environment vs Lexical Environment in V8, and why `let` / `const` behave differently from `var`.

Tricky Code Puzzle

What does this code output, and why?

```
var balance = 10000;

function auditAccount() {
  console.log('Balance before declaration:', balance);
  var balance = 5000;
  console.log('Balance after declaration:', balance);
}

auditAccount();
```

What if `var balance = 5000;` inside the function is changed to `let balance = 5000;`?

Output & Deep Explanation

With `var`:

```
Balance before declaration: undefined
Balance after declaration: 5000
```

With `let`:

```
ReferenceError: Cannot access 'balance' before initialization
```

Hinglish Mental Model

KEY ENGINEERING INSIGHT

"Log sochte hain ki `let` hoist nahi hota. **Galat!** `let` aur `const` bhi hoist hote hain! Farq ye hai ki `var` hoist hote waqt memory me `undefined` se initialize ho jata hai. Lekin `let` aur `const` declaration line tak **Temporal Dead Zone (TDZ)** me rehte hain. TDZ ka matlab: 'Variable memory me allocate ho chuka hai, par uninitialized state me hai. Agar ise touch karoge toh `ReferenceError` throw hoga!'"

Inside `auditAccount()`, the local `let balance` shadows the global `balance`. When `console.log(balance)` executes, JS looks at the local lexical scope, sees `balance` in the TDZ, and throws a `ReferenceError` instead of falling back to the outer global variable (10000).

3. The Classic `var` Inside `setTimeout` Loop Trap

Why Interviewers Ask This

A rite-of-passage interview question testing closures, function-scope vs block-scope, and asynchronous timer callbacks.

Tricky Code Puzzle

What does this print, and how do you fix it in 3 different ways?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 100);
}
```

Intuitive (Wrong) Answer

"0,1,2"

Actual Output

```
3
3
3
```

The Mechanics

`var` is function-scoped (or globally scoped), meaning there is **only one single shared variable** `i` across all loop iterations. By the time the 100ms timer expires and the callback runs, the synchronous loop has already finished, leaving `i = 3`. All 3 callbacks close over the exact same variable reference.

3 Production Fixes

```
// Fix 1: ES6 Block Scope with 'let' (Recommended)
// 'let' creates a brand new binding and lexical scope for every loop iteration
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}

// Fix 2: IIFE (Immediately Invoked Function Expression) - Pre-ES6
for (var i = 0; i < 3; i++) {
  ((capturedIndex) => {
    setTimeout(() => console.log(capturedIndex), 100);
  })(i);
}

// Fix 3: setTimeout Third Argument (Argument Passing)
for (var i = 0; i < 3; i++) {
  setTimeout((capturedIndex) => {
    console.log(capturedIndex);
  }, 100, i);
}
```

4. **this** Keyword Binding Rules & Arrow Function Nuances

Why Interviewers Ask This

this is the most notorious source of bugs in JavaScript. Interviewers check if you know the 4 formal binding rules: 1. Default Binding (standalone function call -> **window** or **undefined** in strict mode) 2. Implicit Binding (called as method of an object -> the object before the dot) 3. Explicit Binding (**call**, **apply**, **bind**) 4. **new** Binding (constructor function -> new instance) 5. **Arrow Functions**: Do NOT have their own **this**; they inherit **this** lexically from their enclosing scope.

Tricky Code Puzzle

What does this print?

```
const bankingService = {
  serviceName: 'IDFC Core UPI',
  getService() {
    return this.serviceName;
  },
  getServiceArrow: () => {
    return this.serviceName;
  }
};

const retrieve = bankingService.getService;

console.log(bankingService.getService());
console.log(retrieve());
console.log(bankingService.getServiceArrow());
```

✓ Output & Explanation

IDFC Core UPI
undefined (or TypeError in strict mode)
undefined

1. `bankingService.getService()` : Implicit binding. `bankingService` is to the left of the dot, so `this` $\equiv$ `bankingService`. Output: `'IDFC Core UPI'`.
2. `retrieve()` : Lost context! `retrieve` is now a standalone function reference. When invoked, Default Binding applies. In non-strict mode `this` is `global/window` (where `serviceName` is undefined); in strict mode it throws `TypeError: Cannot read properties of undefined`.
3. `bankingService.getServiceArrow()` : Arrow functions do **NOT** bind `this` to the object literal! Object literals (`{}`) do NOT create a lexical scope. The enclosing lexical scope is the module/global scope. Output: `undefined`.

⚠ Common Mistake with Event Handlers & React Callbacks

```
// BUG: Arrow function on object prototype
class BankAccount {
  constructor(balance) {
    this.balance = balance;
  }

  // Method tear-off in button click loses context:
  deduct() {
    this.balance -= 100;
  }
}

const acc = new BankAccount(5000);
button.addEventListener('click', acc.deduct); // Crash! 'this' will be the button element, not
BankAccount.

// Fix 1: Bind in constructor
this.deduct = this.deduct.bind(this);

// Fix 2: Class field arrow function
deduct = () => { this.balance -= 100; };
```

5. Type Coercion Brain-Teasers & The IEEE 754 Currency Trap

💡 Why This Matters in Banking

In financial systems, using standard JavaScript floating-point numbers (`Number`) directly for currency arithmetic is a catastrophic bug.

✖ Tricky Floating-Point Trap

```
console.log(0.1 + 0.2 === 0.3); // false!
console.log(0.1 + 0.2);          // 0.30000000000000004
```

🧠 Why This Happens

JavaScript uses the **IEEE 754 double-precision 64-bit binary format**. Numbers like `0.1` (1/10) and `0.2` (1/5) cannot be represented finitely in base-2 binary fractions, resulting in a repeating binary decimal and rounding inaccuracies.

🏦 Production Banking Standard: How to Handle Money in JS

```
// NEVER DO THIS IN BANKING:
const totalAmount = orderItems.reduce((acc, item) => acc + item.price, 0);

// PRODUCTION RULE: Store and compute amounts in PAISE / CENTS (Integers) or use BigInt /
// Decimal.js
// 100.50 INR -> 10050 paise
const amountInPaise = 10050n;
const taxInPaise = 1809n;
const totalInPaise = amountInPaise + taxInPaise; // Precise BigInt math!

// Or safely using integer cents with Math.round:
function addCurrency(a, b) {
  return (Math.round(a * 100) + Math.round(b * 100)) / 100;
}
```

🔗 Classic Type Coercion Rapid-Fire

```
console.log([] + []); // "" (Both arrays convert to empty strings)
console.log([] + {}); // "[object Object]"
console.log({} + []); // 0 in some REPLs (evaluated as block + +[]), or "[object Object]"
console.log(true + false); // 1 (1 + 0)
console.log(+!![]); // 1 (![] -> false, !false -> true, +true -> 1)
console.log('5' - 3); // 2 (Minus operator forces numeric conversion)
console.log('5' + 3); // "53" (Plus operator triggers string concatenation)
console.log(typeof NaN); // "number"
console.log(NaN === NaN); // false (NaN is the only value in JS not equal to itself!)
console.log(Object.is(NaN, NaN)); // true
```

6. Prototypal Inheritance & `__proto__` vs `prototype`

💡 Why Interviewers Ask This

Distinguishes developers who only know ES6 `class` syntax sugar from those who understand JavaScript's underlying prototype chain delegation.

Hinglish Mental Model

KEY ENGINEERING INSIGHT

"**prototype** ek blueprint/cookie-cutter property hai jo sirf **Constructor Functions / Classes** ke paas hoti hai. **__proto__** (internal **[[Prototype]]**) har **object instance** ke paas hota hai jo uske parent prototype object ko point karta hai.

Jab aap **obj.property** access karte ho: 1. JS pehle check karta hai: Kya ye property **obj** ke apne paas hai (**hasOwnProperty**)? 2. Agar nahi, toh wo **obj.__proto__** me check karta hai. 3. Fir uske **__proto__** me, jab tak **null** nahi mil jata. Ise **Prototype Chain** kehte hain!"

```
function BankUser(name) {
  this.name = name;
}

BankUser.prototype.greet = function() {
  return `Hello, ${this.name}`;
};

const user = new BankUser('Bipin');

console.log(user.__proto__ === BankUser.prototype); // true
console.log(BankUser.prototype.__proto__ === Object.prototype); // true
console.log(Object.prototype.__proto__); // null (End of chain)
```

Tricky Question: Why use **Object.create(null)** for in-memory lookup tables?

```
const map1 = {};
console.log('toString' in map1); // true (Inherited from Object.prototype!)

const secureMap = Object.create(null);
console.log('toString' in secureMap); // false (No prototype chain, zero prototype pollution risk!)
```

In banking security, using **Object.create(null)** or **Map** prevents **Prototype Pollution attacks** where an attacker injects malicious properties into **Object.prototype**.

7. Promise Concurrency: `all` vs `allSettled` vs `race` vs `any`

💡 Production Comparison Matrix

Method	Resolves When...	Rejects When...	Ideal Banking Use Case
<code>Promise.all</code>	ALL promises resolve	ANY single promise rejects (Fail-Fast)	Multi-leg transactional checkout: Deduct balance, issue voucher, send ledger notification. All must succeed.
<code>Promise.allSettled</code>	ALL promises settle (resolve OR reject)	NEVER rejects	Bulk statement batch generator or reconciliation: process 5,000 accounts, collect successes and errors separately.
<code>Promise.race</code>	The FIRST promise settles (resolve OR reject)	The FIRST promise settles (resolve OR reject)	Network request timeout wrapper: Race the API call against a 5-second <code>setTimeout</code> rejection.
<code>Promise.any</code>	The FIRST promise resolves successfully	ALL promises reject (<code>AggregateError</code>)	Redundant gateway routing: Ping 3 payment gateways (Razorpay, PayU, BillDesk) simultaneously; first success proceeds.

🧩 Tricky Implementation: Promise Timeout Wrapper

```
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) => {
    setTimeout(() => reject(new Error(`Timeout of ${ms}ms exceeded`)), ms);
  });
  return Promise.race([promise, timeout]);
}

// Usage in UPI authorization:
withTimeout(fetchUPIStatus(txnId), 3000)
  .then(handleSuccess)
  .catch(handleTimeoutOrFailure);
```

8. Object Mutation, Shallow vs Deep Copy & `structuredClone`

💡 The Trap

`Object.assign()` and the spread operator (`...`) perform a **shallow copy**. Nested objects and arrays are shared by reference!

```
const customer = {  
  id: 'CUST-101',  
  preferences: { smsAlerts: true, emailAlerts: false }  
};  
  
const copy = { ...customer };  
copy.preferences.smsAlerts = false;  
  
console.log(customer.preferences.smsAlerts); // false! Original was mutated!
```

🔧 Deep Copying Approaches Compared

1. `JSON.parse(JSON.stringify(obj))` **Traps:**
 2. Drops `undefined`, `Function`, and `Symbol` keys.
 3. Converts `Date` objects into ISO strings.
 4. Converts `NaN` and `Infinity` to `null`.
 5. Throws `TypeError` on circular references!
6. **Native** `structuredClone(obj)` **(Modern Standard):**
 7. Supports circular references, `Date`, `RegExp`, `Map`, `Set`, `ArrayBuffer`.
 8. Does NOT support copying functions or DOM nodes.
9. `Object.freeze()` **Shallow Trap:**
10. `Object.freeze()` only freezes top-level properties. Nested objects can still be mutated unless recursively deep-frozen!

9. Writing Production Debounce & Throttle from Scratch

💡 Why Interviewers Ask This

Directly checks your ability to manage timers, closures, and `this` context without external libraries like Lodash.

Production Debounce (with Immediate / Leading Execution Option)

```
function debounce(fn, delay, immediate = false) {
  let timerId = null;

  const debounced = function(...args) {
    const context = this;
    const callNow = immediate && !timerId;

    clearTimeout(timerId);

    timerId = setTimeout(() => {
      timerId = null;
      if (!immediate) fn.apply(context, args);
    }, delay);

    if (callNow) {
      fn.apply(context, args);
    }
  };

  debounced.cancel = function() {
    clearTimeout(timerId);
    timerId = null;
  };

  return debounced;
}

// Banking Use Case: Search Beneficiary IFSC Code as user types
const searchIFSC = debounce((query) => fetchBankBranch(query), 300);
```

Production Throttle (Guaranteed Rate Limiting)

```
function throttle(fn, limit) {
  let inThrottle = false;
  let lastFn = null;
  let lastTime = 0;

  return function(...args) {
    const context = this;
    const now = Date.now();

    if (!inThrottle) {
      fn.apply(context, args);
      lastTime = now;
      inThrottle = true;

      setTimeout(() => {
        inThrottle = false;
        if (lastFn) {
          lastFn();
          lastFn = null;
        }
      }, limit);
    } else {
      lastFn = () => fn.apply(context, args);
    }
  };
}

// Banking Use Case: Prevent double-tapping payment authorization buttons
const submitPayment = throttle(initiateTransfer, 2000);
```

10. V8 Engine Optimization: Hidden Classes & Inline Caches

💡 Why Senior Interviewers Ask This

To evaluate whether you understand how high-performance JavaScript executes under the hood at machine level in the V8 engine (Node.js & Chrome).

🧠 The Mechanics

JavaScript is dynamically typed, so property lookups in objects normally require an expensive dictionary hash search. V8 optimizes this by creating internal **Hidden Classes (Shapes)**: 1. Every object points to a Hidden Class describing its property offsets in memory. 2. When two objects share the exact same properties added in the **exact same order**, V8 shares the Hidden Class. 3. **Inline Caching (IC)**: If a function repeatedly accesses properties of objects with the same Hidden Class (Monomorphic), V8 replaces the lookup with a direct memory offset jump (C++ struct speed).

🚨 How Developers Accidentally De-Optimize V8

```
// BAD: Adding properties in different orders creates diverging Hidden Classes!
function createUserA() {
  const u = {};
  u.id = 1;
  u.name = 'Bipin';
  return u;
}

function createUserB() {
  const u = {};
  u.name = 'Bipin'; // Different order!
  u.id = 1;
  return u;
}

// BAD: Deleting properties using 'delete' forces the object into Slow Dictionary Mode!
delete user.id; // Forces V8 to abandon Hidden Class optimizations

// GOOD: Initialize all fields in the constructor, set unused fields to null
class Account {
  constructor(id, pan = null) {
    this.id = id;
    this.pan = pan; // Consistent shape guaranteed
  }
}
```

⚡ Rapid-Fire JavaScript Interview Puzzles

1. What is the output of `typeof null` ?

`"object"`. This is a legacy bug from JS 1.0 where type tags stored `000` for objects and `null` was represented as the NULL pointer (`0x00`), mistakenly matching the object tag.

2. What does `[] = ![]` evaluate to?

`true`!

Step 1: `![]` converts to boolean `false`.

Step 2: `[] = false`.

Step 3: Boolean converts to number: `[] = 0`.

Step 4: Array converts to primitive string `""`: `"" = 0`.

Step 5: String converts to number: `0 = 0` -> `true`!

3. Why does `[1, 2, 10].sort()` result in `[1, 10, 2]`?

Default `.sort()` converts elements to strings and compares UTF-16 code units. You must provide a comparator: `.sort((a, b) => a - b)`.

4. Can you modify a property of an object declared with `const`?

Yes. `const` creates an immutable variable binding to the memory address, not an immutable object value. To make the object immutable, use `Object.freeze()`.

CHAPTER 02 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Drops ``undefined``, ``Function``, and ``Symbol`` keys.
- Converts ``Date`` objects into ISO strings.
- Converts ``NaN`` and ``Infinity`` to ``null``.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

Node.js Event Loop, libuv & Concurrency Internals

PRIORITY

● High Priority

DIFFICULTY

Candidate Superpower

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Node.js Event Loop, libuv & Concurrency Internals

1. Why This Matters

As a 3+ YoE developer whose strongest professional background is in Node.js and Express, this is your primary architectural superpower. At IDFC FIRST Bank's Strategic Projects, Node.js microservices act as high-throughput API gateways and orchestrators for core banking services. An interviewer will not ask basic definition questions; they will grill you on: – Exactly how the **libuv event loop phases** operate in C++. – The precise priority difference between `process.nextTick()` and the V8 Promise Microtask Queue. – Where and how blocking operations (DNS lookups, `fs`, `crypto`) run on the **libuv worker thread pool**. – How CPU-bound tasks cause **event loop lag**, spiking the p99 latency of financial API endpoints.

2. Prerequisites

- Familiarity with asynchronous JavaScript, Callbacks, Promises, and `async/await`.
- Understanding of the single-threaded JavaScript execution model in Google V8.

3. Concept

Architecture Overview

Node.js consists of: 1. **V8 Engine**: Compiles and executes JavaScript code, manages the Call Stack and Memory Heap. 2. **libuv (C library)**: Provides the cross-platform Event Loop, non-blocking asynchronous I/O (via OS primitives like `epoll` on Linux, `kqueue` on macOS, `IOCP` on Windows), and a 4-thread worker pool (`UV_THREADPOOL_SIZE`).

flowchart TD

```
subgraph V8["V8 Engine"]
  CS["Call Stack (Synchronous Code)"]
  Heap["Memory Heap (Objects, Closures)"]
end
```

```
subgraph Queues["Microtask Queues (Between Every Phase)"]
  NT["process.nextTick Queue (Highest Priority)"]
  PM["Promise Microtask Queue (.then, await)"]
end
```

```
subgraph EventLoop["libuv Event Loop Phases (Macrotasks)"]
  P1["1. Timers (setTimeout, setInterval)"]
  P2["2. Pending Callbacks (I/O exceptions, TCP errors)"]
  P3["3. Idle / Prepare (Internal libuv use)"]
  P4["4. Poll (Retrieve I/O events: epoll_wait)"]
  P5["5. Check (setImmediate callbacks)"]
  P6["6. Close Callbacks (socket.on('close'))"]
end
```

```
subgraph ThreadPool["libuv Worker Thread Pool (Default: 4 Threads)"]
  T1["fs (File System)"]
  T2["crypto (pbkdf2, scrypt)"]
  T3["dns.lookup (OS resolver)"]
  T4["zlib (Compression)"]
end
```

CS → Queues

Queues → EventLoop

EventLoop → ThreadPool

4. Simple Example: Predicting Execution Order

```
// Guess the exact order before reading the solution
console.log('1. Sync Start');

setTimeout(() => {
  console.log('2. setTimeout 0');
}, 0);

setImmediate(() => {
  console.log('3. setImmediate');
});

Promise.resolve().then(() => {
  console.log('4. Promise Microtask');
});

process.nextTick(() => {
  console.log('5. process.nextTick');
});

console.log('6. Sync End');

/*
Output Order:
1. Sync Start
6. Sync End
5. process.nextTick      (Microtask - nextTick queue executes immediately after sync code)
4. Promise Microtask     (Microtask - Promise queue executes after nextTick queue)
2. setTimeout 0          (Timers phase)
3. setImmediate          (Check phase)
*/
```

5. Real-World Banking Example: The Event Loop Lag Outage

In a banking microservice generating account statement PDFs: - An engineer uses a synchronous encryption loop or large JSON parse (`JSON.parse(hugePayload)`) on the main thread. - While the CPU core executes the 800ms parsing loop, the **Call Stack is completely blocked**. - Incoming UPI webhook confirmations cannot enter the Poll phase; their TCP sockets time out. - Client applications experience cascading 504 Gateway Timeouts, even though the database is healthy.

Architectural Fix: Offload CPU-heavy encryption or PDF generation to **Worker Threads** (`worker_threads`) or a dedicated asynchronous background worker (BullMQ / RabbitMQ), keeping the event loop idle to process fast I/O requests.

6. Code: Verifying `setImmediate` vs `setTimeout` Inside I/O Cycle

```
const fs = require('fs');

fs.readFile(__filename, () => {
  // We are now inside the Poll phase callback of an I/O cycle!
  console.log('--- Inside I/O Callback (Poll Phase) ---');

  setTimeout(() => {
    console.log('1. setTimeout (Timers Phase - next loop tick)');
  }, 0);

  setImmediate(() => {
    console.log('2. setImmediate (Check Phase - runs FIRST!)');
  });

  process.nextTick(() => {
    console.log('0. process.nextTick (Runs immediately after this callback)');
  });
});

/*
Guaranteed Output Inside I/O Cycle:
--- Inside I/O Callback (Poll Phase) ---
0. process.nextTick
2. setImmediate (Check Phase - runs FIRST!)
1. setTimeout (Timers Phase - next loop tick)
*/
```

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Crucial Interview Insight: In the main module scope, `setTimeout(..., 0)` vs `setImmediate` is non-deterministic (depends on process startup clock jitter). But **inside an I/O callback**, `setImmediate` is 100% guaranteed to run before `setTimeout` because the event loop moves directly from the **Poll phase** to the **Check phase**!

7. How It Works Internally

1. libuv Event Loop Loop Iteration (C++ Pseudo-Code)

```
while (r = 0 && loop->stop_flag == 0) {
  uv__update_time(loop);
  uv__run_timers(loop);           // 1. Timers Phase
  uv__run_pending(loop);         // 2. Pending I/O callbacks
  uv__run_idle(loop);
  uv__run_prepare(loop);
  uv__io_poll(loop, timeout);    // 4. Poll Phase (blocks waiting for epoll)
  uv__run_check(loop);          // 5. Check Phase (setImmediate)
  uv__run_closing_handles(loop); // 6. Close callbacks
}
```

2. The Poll Phase Mechanics

When the loop enters the **Poll phase**: 1. It calculates how long it should block waiting for I/O: – If there are `setImmediate` callbacks scheduled, timeout is `0` (advances immediately to Check phase). – If there are timers scheduled, timeout is `timer_due_time - current_time`. – If no timers exist, it blocks indefinitely waiting for incoming socket/file events. 2. Once events arrive via `epoll_wait()`, it executes their callbacks.

3. Microtask Drain Rules (Node.js 11+)

In Node.js 11+, the microtask queue is drained **immediately after every single callback** executes, matching browser behavior, rather than waiting for the entire phase queue to finish.

8. Common Mistakes

1. **Believing `setTimeout(fn, 0)` Runs in Exactly 0 Milliseconds:** The minimum timer resolution in libuv and OS kernels is typically 1ms. If the event loop is busy, it may run after 50ms.
2. **Confusing `process.nextTick()` with `setImmediate()`:**
3. `process.nextTick()` is **not** part of the libuv event loop. It belongs to the Node.js process microtask queue and runs immediately after the current operation finishes, before the event loop continues. Recursive `process.nextTick()` starves the event loop entirely!
4. `setImmediate()` runs during the libuv **Check phase**.
5. **Believing All Async Node.js Operations Use the Thread Pool:** Network I/O (`http`, `https`, `net`, `tls`) does **NOT** use the 4-thread pool! Network sockets use native OS kernel async notifications (`epoll` / `kqueue`). The thread pool is reserved for `fs`, `crypto`, `zlib`, and `dns.lookup`.

9. Performance / Complexity Matrix

Operation	Execution Mechanism	Blocks Main Event Loop?	Thread Pool Usage?
Incoming HTTP Request	OS kernel <code>epoll</code> / non-blocking	No	No (OS native)
<code>crypto.pbkdf2</code>	libuv worker thread	No (Async API)	Yes (Consumes 1 thread)
<code>fs.readFile</code>	libuv worker thread	No (Async API)	Yes (Consumes 1 thread)
<code>JSON.parse(100MB)</code>	V8 Call Stack (Synchronous)	YES (FREEZES SERVER)	No
RegExp with Catastrophic Backtracking	V8 Call Stack	YES (CPU SPIKE)	No

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** Is Node.js completely single-threaded? Explain what parts are single-threaded and what parts are multi-threaded.

Medium

- **Q:** What is the difference between `process.nextTick()` and `setImmediate()`? Why can recursive `process.nextTick()` freeze an application while recursive `setImmediate()` does not?

Hard

- **Q:** A Node.js backend handles 5,000 requests/sec for lightweight database lookups. An engineer adds an async password hashing call using `bcrypt.hash()` or `crypto.pbkdf2()` with default settings on login. Suddenly, database request latencies explode across all endpoints. Explain why this happens under the hood and how to fix it.
-

11. Follow-up Questions from Interviewer

- "What is `UV_THREADPOOL_SIZE` and what is its default value? What are the dangers of increasing it to 128?" (Answer: Default is 4, max is 1024. Increasing it too high causes severe OS thread context-switching overhead and memory thrashing on multicore CPUs).
 - "What is the difference between `dns.lookup` and `dns.resolve` in Node.js?" (Answer: `dns.lookup` calls the synchronous OS `getaddrinfo(3)` system call, using a libuv worker thread! `dns.resolve` uses asynchronous c-ares network queries, bypassing the thread pool entirely).
-

12. Model Answer: libuv Thread Pool Starvation Under Crypto Load

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

Interviewer: "Why did adding async password hashing cause unrelated database queries and file reads to stall in our Node.js microservice?"

Model Answer: "While JavaScript executes on a single thread, Node.js offloads specific operations—namely `fs`, `crypto`, `zlib`, and `dns.lookup`—to the **libuv worker thread pool**, which has a default size of **4 threads**.

When multiple concurrent login requests arrive, each `crypto.pbkdf2` or `bcrypt` call acquires one of the 4 worker threads for CPU-intensive key derivation (taking perhaps 80–100ms per hash).

If 4 hashing tasks are active simultaneously, the thread pool is **completely saturated**.

When an incoming database request requires an internal file system read, or DNS resolution (`dns.lookup`), or TLS certificate validation, those tasks are queued in the libuv thread pool queue waiting for a free worker. Even though the main V8 event loop is idle, I/O tasks are blocked in the worker queue, causing response latencies to spike across unrelated endpoints.

Solution: 1. Increase `UV_THREADPOOL_SIZE` on startup (e.g., `process.env.UV_THREADPOOL_SIZE = Math.min(128, os.cpus().length * 2)`). 2. Offload CPU-bound authentication hashing to dedicated authentication microservices or worker threads (`worker_threads`), preventing crypto workloads from competing with core I/O."

13. Practical Exercise

Create and execute a script that proves `UV_THREADPOOL_SIZE = 4` by launching 6 concurrent `crypto.pbkdf2` calls and measuring the execution timestamps:

```
// threadpool_test.js
const crypto = require('crypto');
const start = Date.now();

function hashPassword(id) {
  crypto.pbkdf2('secret_password', 'salt', 100000, 64, 'sha512', () => {
    console.log(`Task ${id} completed in ${Date.now() - start}ms`);
  });
}

for (let i = 1; i <= 6; i++) {
  hashPassword(i);
}

// Notice that Tasks 1-4 finish almost simultaneously, while Tasks 5-6 finish ~2x later!
```

14. Quick Revision

- V8 handles JS execution; libuv handles OS event loop & thread pool.
- Event loop phases: Timers → Pending → Poll → Check → Close.

- Microtasks (`nextTick` , `Promise`) run *immediately* after the active stack frame empties, before the next event loop phase.
- Network sockets do **not** use the thread pool (`epoll` handles non-blocking socket notifications).
- Thread pool size default is 4; handles `fs` , `crypto` , `zlib` , and `dns.lookup` .

15. Interview Checklist

- [] Can draw the libuv event loop phase diagram from memory.
- [] Explains `process.nextTick` vs `setImmediate` with precision.
- [] Understands why `setImmediate` runs before `setTimeout` inside an I/O callback.
- [] Explains why network I/O does not consume libuv worker threads.
- [] Identifies thread pool starvation and knows how to configure `UV_THREADPOOL_SIZE` .

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Exactly how the libuv event loop phases operate in C++.
- The precise priority difference between `process.nextTick()` and the V8 Promise Microtask Queue.
- Where and how blocking operations (DNS lookups, `fs` , `crypto`) run on the libuv worker thread pool.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

Node.js & Backend Architecture Last-Minute Cheatsheet

PRIORITY

 High Priority

DIFFICULTY

Candidate Superpower

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Node.js & Backend Architecture Last-Minute Cheatsheet

libuv Event Loop Phase Order

Synchronous Call Stack (V8)



[Microtask Drain: process.nextTick → Promise Microtasks]


 1. Timers Phase (setTimeout, setInterval callbacks)
 [Microtask Drain]

 2. Pending I/O Phase (System/TCP error callbacks)
 [Microtask Drain]

 3. Idle / Prepare (libuv internal use)
 [Microtask Drain]

 4. Poll Phase (Retrieve I/O events via epoll/kqueue; executes I/O callbacks)
 [Microtask Drain]

 5. Check Phase (setImmediate callbacks)
 [Microtask Drain]

 6. Close Callbacks (socket.on('close'))
 [Microtask Drain]

Key Differences & Interview Gotchas

1. `process.nextTick()` vs `setImmediate()` :
2. `process.nextTick()` is **not** part of libuv; executes immediately after current synchronous code. Infinite recursion freezes the server!
3. `setImmediate()` runs during the libuv **Check phase**.
4. Inside an I/O callback, `setImmediate` **always runs before** `setTimeout(0)` .
5. **What uses the libuv Worker Thread Pool (`UV_THREADPOOL_SIZE = 4`)?**
6. **Uses Thread Pool:** `fs` (file system), `crypto` (`pbkdf2` , `scrypt`), `zlib` (compression), `dns.lookup` .

7. **Does NOT use Thread Pool:** Network sockets (`http` , `https` , `tls` , `net`), `dns.resolve` (handled non-blocking via OS `epoll` / `kqueue`).

8. **Streams & Backpressure:** ``javascript // Anti-Pattern (Leaks RAM when client network is slow):
fs.readFile('huge.csv', (err, data) => res.send(data));

// Production Pattern with Backpressure (Stream pipeline): const { pipeline } = require('stream/promises'); await pipeline(fs.createReadStream('huge.csv'), res); ``

Memory Leak Detection in Node.js

- **Tool:** Chrome DevTools (`node --inspect`) or `v8.writeHeapSnapshot()` .
- **Top 3 Culprits:**
 - Unbounded global caching objects without TTL/eviction.
 - Unremoved event listeners (`emitter.on` inside request handlers).
 - Stale closures retaining large outer function scope objects.
- **Metric to watch:** `process.memoryUsage().heapUsed` vs `heapTotal` .

Clustering vs Worker Threads

Dimension	Cluster Module (<code>cluster</code>)	Worker Threads (<code>worker_threads</code>)
Model	Multi-Process (Forking)	Multi-Thread within single process
Memory	Isolated memory space per core	Shared memory via <code>SharedArrayBuffer</code>
Port Sharing	Shares same server port via master proxy	Run within existing master process
Best Used For	Scaling web servers across CPU cores	Heavy CPU tasks (image resize, crypto)

CHAPTER 04 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- `process.nextTick()` is not part of libuv; executes immediately after current synchronous code. Infinite recursion freez
- `setImmediate()` runs during the libuv Check phase.
- Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)`.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases